
ENGINEERING CHANGE DOCUMENT

DataProvider (Views) — Change Document

A new tabbed variant of the Elbrit DataProvider, and the additive changes made to the existing data engine to support it.

SCOPE	<code>share/src/app/datatable/</code> + <code>share/src/plasmic-init.js</code>
COMMIT RANGE	<code>62ee1ee..dd84b57</code> (base: <code>f8e3939</code>)
NET EFFECT	7 new files, 4 existing files modified, 1 file added-then-removed.

Contents

1. Why this work happened

The governing decision

2. Architecture

3. New files

- 3.1 contexts/ViewContext.jsx (15 lines)
- 3.2 components/DataView.jsx (43 lines)
- 3.3 components/DataProviderViews.jsx (289 lines) — the variant
- 3.4 components/views/ProductSearchBar.jsx (191 lines)
- 3.5 components/views/AlphabetRail.jsx (225 lines)
- 3.6 components/views/SyncPill.jsx (126 lines)
- 3.7 components/views/FilterSortPill.jsx (61 lines)
- 3.8 components/views/StaleDataBridge.jsx (189 lines)
- 3.9 Added then removed: views/SortSheet.jsx

4. Modified existing files

- 4.1 components/DataProviderNew.jsx (+56 / -5)
- 4.2 components/FilterSortSidebar.jsx (+15 / -4)
- 4.3 plasmic-init.js (+203 / -2)
- 4.4 config/configs/slotConfig.js (+3 / -3)

5. Plasmic Studio prop reference

Elbrit DataProvider (Views)

Elbrit DataView

Studio bindings

6. Backward compatibility

7. Known limitations and gotchas

8. Commit map

Companion work outside share/ (context only)

1. Why this work happened

The existing `DataProvider` (Plasmic: *Elbrit DataProvider*) is a single-slot component. It fetches once, owns one filter/sort/search state, and publishes everything on `$ctx.data`. That works well for one table on a page.

The product ask was a **mobile product-catalog screen** with:

- a **Cards view and a Table view of the same data**, switchable by a toggle;
- a **search bar** that filters both views at once;
- an **A–Z jump rail** (iOS-contacts style) down the side of the cards;
- a **compact, mobile-sized control row** instead of the desktop-scale header;
- **instant paint on repeat visits** rather than a spinner every time.

None of this fits the plain `DataProvider`, and all of it could have been bolted onto `DataProviderNew` — which is the risk. `DataProviderNew.jsx` is ~5,900 lines and backs **every table in the app**. Any change to its fetch loop, loading flow, or header rendering is a blast-radius change.

The governing decision

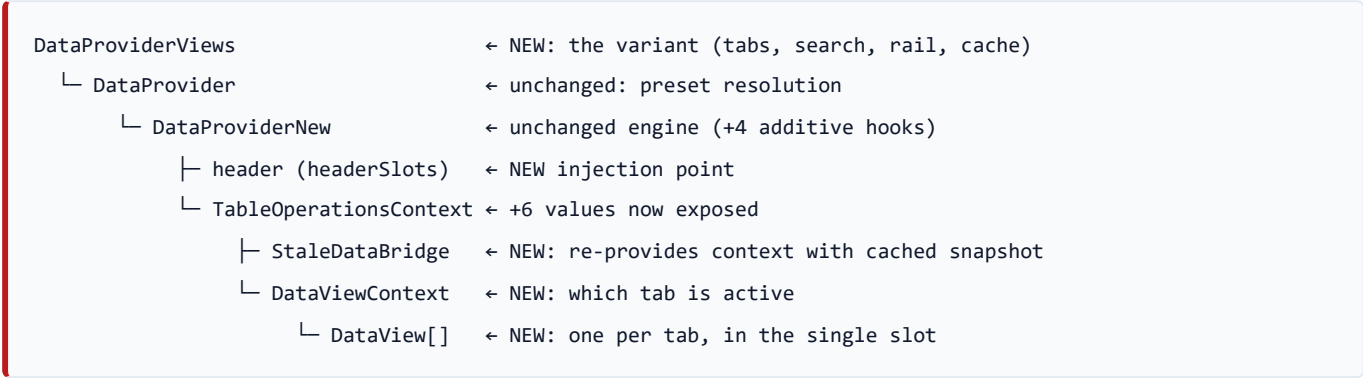
Build a sibling variant that wraps the existing engine. Touch the engine only to *expose* things it already had, never to change what it does.

So:

- `DataProviderViews` **wraps** `DataProvider` — same preset resolution, same fetch, same filter/sort state. There is no second data path to keep in sync.
- Every new behaviour (tabs, search bar, letter rail, compact pills, stale cache) lives in **new files under `components/views/`**, consuming the engine through the existing `TableOperationsContext`.
- The four edits to existing files are **purely additive and default-off**. With no new props passed, every current page renders byte-identically.

That constraint is why the code looks the way it does in several places, and it is called out again at each edit below.

2. Architecture



Two context layers, deliberately separate:

Context	Owner	Carries	Why separate
TableOperationsContext	DataProviderNew	data, columns, filter/sort/search, sync	Pre-existing. The variant only reads it.
DataViewContext	DataProviderViews	views, activeView, setActiveView, isActive	View state is a variant concern. The engine must not learn about tabs.

Both are also published to Plasmic Studio: `$ctx.data` (existing) and `$ctx.view` (new).

3. New files

3.1 `contexts/ViewContext.jsx` (15 lines)

Plain React context + `useDataViews()` hook.

Why it returns `null` outside the provider: so a `DataView` dropped under the *plain* `DataProvider` degrades to always-visible instead of crashing. Studio users move components between providers; that should not produce a blank screen.

3.2 `components/DataView.jsx` (43 lines)

One tab's content. Renders children only when its `viewId` is active.

Two decisions worth recording:

`display: contents` by default. A `DataTableNew` needs an unbroken flex chain from the provider down to itself to compute its height. A normal `<div>` wrapper breaks that chain and the table collapses. `display: contents` makes the wrapper layout-transparent — the child participates in the *grandparent's* flex context. Passing `className` or `style` opts back into a real box when you actually want one.

Hidden, not unmounted (`keepInactiveMounted` , default `true`). Unmounting an inactive view throws away the child's local state — table scroll position, expanded rows, column widths. Tab away and back and the user loses their place. So inactive views get `display: none` and stay mounted. `keepMounted={false}` opts out per view when the memory cost matters more.

ARIA `role="tabpanel"` / `aria-labelledby` are wired to the switcher's `role="tab"` ids.

3.3 `components/DataProviderViews.jsx` (289 lines) — the variant

Wraps `DataProvider` , owns view state, and assembles the header.

`normalizeViews()`

Accepts `['Cards', 'Table']` or `[{id,label,icon}]` , de-dupes by id, falls back to a Cards/Table default. **Why:** Studio users type an array of strings; code callers want icons. Rejecting either form would be a support burden for zero benefit.

Controlled / uncontrolled `activeView`

Uncontrolled by default (internal `useState`); becomes controlled the moment `activeView` is non-empty. Registered in Plasmic as a **writable state**, so Studio can either let the provider own the tab or drive it from a page variable.

A `useEffect` re-validates the internal selection when the `views` list changes underneath it, and `resolvedActiveView` falls back to the first view if a controlled value names a tab that does not exist — **an unknown id paints the first view rather than an empty screen.**

Header assembly — and why every piece is `useMemo` 'd

The header elements are passed down through `__internal.headerSlots` , which lands in a `useMemo` dependency array inside `DataProviderNew` . Inline JSX would produce a new object identity on every render and defeat that memo, re-rendering the whole engine on every keystroke. Hence `switcher` , `headerTop` , `headerLeft` are each memoized.

Compact mode (`compactHeader` , defaults to `showSearch`)

Sets `__internal.showProviderHeader = false` and renders the variant's own pills instead:

Why everything goes in the LEFT slot as one `justify-between` row: the engine's header row is `flex-col sm:flex-row`. Below the `sm` breakpoint it *stacks* left and right slots vertically — which is exactly the wrong thing on the mobile screen this was built for. Putting the whole row in the left slot keeps it on one line at every width. `flex-nowrap` + `min-w-0` enforce it.

Sizing: pills and switcher are all `height: 1.75rem`, against the engine's own `2rem` controls — deliberately one notch smaller, since compact mode never shows them side by side.

Spacing constants

`DEFAULT_CONTENT_PADDING` and `HEADER_SLOT_PADDING` exist because the engine's header wrapper is `px-2` on mobile, which reads as cramped. **The variant insets its own slots rather than changing the engine's padding** — same constraint as everywhere else.

Letter rail placement

The rail is rendered as a **sibling** of the children slot inside a constant flex row, and gated by `letterRailViews` (default `['cards']`) — the table view has no letter sections).

Why the wrapper row is constant: if the row itself appeared and disappeared with the active view, React would remount the slot content on every tab switch, defeating `keepInactiveMounted`. Only the rail toggles; the row does not.

Stale-while-revalidate wiring

When `staleWhileRevalidate` is on, the content is wrapped in `StaleDataBridge` with a key defaulting to `preset:{presetDataSource}:{presetName}`.

3.4 components/views/ProductSearchBar.jsx (191 lines)

Search input that drives the engine's own `setSearchTerm`, so **cards and table filter together from one state** — that was the requirement, and it is why this drives the provider rather than filtering locally.

Detail	Why
250 ms debounce on <code>setSearchTerm</code> , immediate on the local <code>text</code> state	The provider re-filters the whole dataset on every change. Typing must stay responsive without re-filtering per keystroke.
<code>setSearchTermRef</code> instead of the callback in deps	The engine's <code>setSearchTerm</code> identity is not guaranteed stable; a ref keeps the debounce closure from being rebuilt mid-type.
Sync-back <code>useEffect</code> on <code>searchTerm</code>	Something else (e.g. <code>clearAllFilters</code>) can reset the term. Without this the input would show a stale query.
Recents panel is <code>position: fixed</code> , anchored to the input's rect	The engine's header sets <code>overflow-x-auto</code>, which clips the cross axis too. An absolutely positioned panel gets cut off. Fixed positioning escapes the clipping context entirely. Repositioned on scroll/resize.
Every <code>localStorage</code> access wrapped in <code>typeof window + try/catch</code>	SSR/prerender has no <code>localStorage</code> ; private mode throws on quota. Recents are a convenience, never a failure path.
<code>searchUnavailable</code> → a <code>title</code> tooltip, not a disabled input	The underlying search only works when the query doc has <code>clientSave: true</code> and a <code>searchFields</code> map. Silently doing nothing was the previous failure mode; this explains it.

3.5 components/views/AlphabetRail.jsx (225 lines)

The A–Z jump rail. **Owned by the provider, not by any card component** — so it can be toggled from one prop and works with any Studio-built layout.

The DOM contract

Clicking "A" scrolls to the nearest `[data-letter="A"]` inside the provider's content area. That is the entire coupling. `CatalogLetterGroup` renders these attributes; any custom Studio layout can too. **Why a DOM contract rather than a data prop:** the rail must not know how the slot content is structured, and the slot content is built in Studio where we cannot enforce a component API.

Two sources of letters

- `letterRailField` set (e.g. `brand__name`) — letters come from the provider's own pipeline data, so **search and filter dim letters live**.
- **No field** — a `MutationObserver` learns the letters from whatever `[data-letter]` sections rendered.

`readField()` tolerates flattened (`brand__name`), nested (`brand.name`), and scalar-Link shapes, because rows arrive in all three depending on the query.

Why a rAF-throttled scroll listener, not `IntersectionObserver`

This was a considered choice. `IntersectionObserver` is the idiomatic answer but is unreliable here:

- sections taller than the viewport never intersect cleanly;
- several sections can share a letter;
- sections mount *after* the rail does.

The implementation instead re-queries `[data-letter]` each frame and picks the section spanning a probe line at 20% viewport height, falling back to the nearest section top above it. Deterministic in all three cases. The listener is registered with `capture: true` so it catches scrolls of **nested** containers, not just the window.

Scrubbing

Pointer events + `elementFromPoint` give iOS-contacts-style drag-to-scrub, with a floating letter bubble.

`touch-action: none` stops the page panning under the finger. Jumps during a drag use `behavior: 'auto'` — **smooth scrolling cannot keep up with a finger** and lags visibly behind. Taps still use smooth via the buttons' `onClick`.

3.6 components/views/SyncPill.jsx (126 lines)

Compact replacement for the engine's sync `SplitButton` in compact mode: `🕒 5 Aug, 11:25 ▾`, with *Hard Refresh* on the chevron.

It calls the engine's own `handleSync` / `handleHardRefresh`. Nothing about refreshing was reimplemented — only the button was. This is why those two functions had to be exposed on the context (§4.1).

- `busy` ORs `executingQuery || isRevalidating || isLoading` — `executingQuery` **alone misses the cache-read phases of a sync**, so the spinner would stop early and the control would look idle mid-refresh.
- Renders `null` when there is no `dataSource` (offline data) — nothing to sync.
- Menu is `position: fixed` anchored to the button, for the same `overflow-x-auto` clipping reason as the search bar; it closes on scroll/resize so it cannot drift away from its trigger.

3.7 components/views/FilterSortPill.jsx (61 lines)

Compact trigger: `⚙️ A → Z`.

This is only a restyled button. It opens the *original* `FilterSortSidebar`, with `sortFields` / `searchFields` from the same query doc. There is no parallel sort implementation.

By default it opens the sidebar in **sort-only** mode, because in this layout the search bar owns filtering and the filter tabs would be redundant.

The label is typed the same way the engine's own applied-sort chip types it — `A → Z` for text, `Low → High` for numbers, `Oldest → Latest` for dates — so the two surfaces never disagree.

It mirrors the native button's availability check exactly (`clientSave === true` and search or sort fields present), so the pill can never appear where the sidebar would be empty.

3.8 components/views/StaleDataBridge.jsx (189 lines)

Stale-while-revalidate: paint last session's data instantly, refresh behind it.

The central point: `DataProviderNew`'s **loading flow is not modified**. The bridge sits *between* the provider and the slot content and works purely on the published context value:

1. After each successful load it snapshots the **data fields only** to IndexedDB (`elbrit-view-snapshots`).
2. On the next visit, while the provider is loading and has nothing on screen, it re-provides the context with the snapshot patched in — `isLoading: false`, `isRevalidating: true`.
3. The moment live data lands it passes the real context through untouched and re-snapshots.

Details and their reasons:

Detail	Why
<code>SNAPSHOT_FIELDS</code> is an explicit allow-list of 12 data fields	Functions are never snapshotted. The live provider's callbacks are kept, so sort/filter/sync still work during the stale window instead of calling into a dead cached closure.
<code>MAX_SNAPSHOT_ROWS = 20000</code>	The goal is a fast first paint, not a full offline mirror. Serializing a huge dataset costs more than the spinner it saves.
<code>snapshotSignature()</code> before writing	Cheap change detection so identical data isn't rewritten to IndexedDB on every render.
<code>idbSet</code> failure retries without <code>reportData</code>	<code>reportData</code> can hold non-structured-cloneable values. Rather than losing the whole snapshot, it drops the one risky field.
<code>normalizeSlots()</code> handles flat vs slot-map contexts	The engine publishes <code>{rawData,...}</code> in some configurations and <code>{main:{...}}</code> in others. Both must snapshot.
Providers always render, even when not patching	If the provider element itself appeared/disappeared, the entire child tree would remount when patching toggled — a visible flash exactly when we're trying to avoid one.
<code>shouldPatch = anyLoading && !hasLiveData && snapshot != null</code>	Patch only during a cold load with nothing on screen. Never overwrite live data with stale data.

First-ever visit is a passthrough — the normal spinner shows. Documented as *avoid on tables where users edit rows*, since a stale picture of editable data is misleading.

3.9 Added then removed: `views/SortSheet.jsx`

Added in `7553e84`, deleted in `40f5216`.

The first attempt at compact sorting was a **bottom sheet of named sort presets** driving `updateSort` with PrimeReact `sortMeta`. It was dropped because it was a **second sort implementation** that had to be configured separately from the query doc's `sortFields` — two places to maintain, and two sources of truth that could disagree.

Replaced by `FilterSortPill` + sort-only mode on the existing sidebar: same compact trigger, zero duplicated logic. This is the clearest example of the governing decision in §1 being applied mid-stream.

4. Modified existing files

Every change below is additive and default-off.

4.1 `components/DataProviderNew.jsx` (+56 / -5)

Four changes, no behavioural change to existing callers.

(a) `__internal.headerSlots` — **caller-supplied header content**. New prop `{top, left, right}`, default `null`.

- `top` — full-width row above the control row (the search bar).
- `left` / `right` — bracket `selectorsJSX` inside its existing `justify-between` row.

The header wrapper condition widened from `hasHeaderContent` to `hasHeaderContent || headerSlotTop || headerSlotLeft || headerSlotRight`, and `selectorsJSX` is now gated on `hasHeaderContent`. **With `headerSlots` null all three expressions collapse to the original behaviour** — the header renders iff `hasHeaderContent`, containing exactly `selectorsJSX`.

Why in the engine at all: the variant needs its controls **inside** the engine's header (sharing its border, background, and sticky behaviour), not in a second bar above it. Two stacked bars looked wrong and wasted vertical space on mobile.

(b) `__internal.hideNativeFilterSort` — drops the built-in *Filter / Sort* button when the caller supplies its own.

Applied-filter chips are kept. Default `false`.

(c) **Sort-only sidebar access**. New state `filterSortSidebarSortOnly` + `openFilterSortSidebar({sortOnly})`, passed to `FilterSortSidebar` as `sortOnly`. **The native button now explicitly sets `sortOnly = false`** before opening, so the flag can never leak from a pill-opened session into a button-opened one.

(d) **Six values added to `TableOperationsContext`** (both the main and the secondary context builder, with matching dependency-array entries):

```
handleSync, handleHardRefresh, lastUpdatedAt, formatLastUpdatedDate,  
setFilterSortSidebarVisible, openFilterSortSidebar
```

Why: `SyncPill` and `FilterSortPill` must invoke the **existing** behaviours. Exposing them was the alternative to reimplementing them — which is what makes those two components ~60 and ~126 lines instead of several hundred each.

4.2 `components/FilterSortSidebar.jsx` (+15 / -4)

New `sortOnly` prop, default `false`.

- Hides the left tab-navigation column (sort-only has a single pane) and forces `activeTabIndex = 0` on open.
- Header text becomes `Sort` instead of `Filter and Sort`.
- `handleClear` **returns early before touching filter selections** — clearing sort must not silently wipe filters the search bar or another surface applied.
- `hasActiveFilters` considers only the sort in this mode.

Default `false` preserves the original behaviour exactly.

4.3 `plasmic-init.js` (+203 / -2)

Registers `DataProviderViews` (*Elbrit DataProvider (Views)*) and `DataView` (*Elbrit DataView*, `parentComponentName: 'DataProviderViews'`) in the `ElbritCoreLib` section, and adds both to the components map and the export list.

`DataProviderViews` declares `providesData: true`, a **writable** `activeView` **state**, and a default children slot pre-filled with a `cards` and a `table` `DataView` (the table one already containing a `DataTableNew`) — so dragging it in produces a working two-tab setup without manual assembly.

The prop descriptions are written as **Studio-facing documentation**, including the constraints a Studio user cannot discover from the UI: that search needs `clientSave: true` + `searchFields`, that `compactHeader` hides the month picker, that `staleWhileRevalidate` should be avoided on editable tables.

4.4 `config/configs/slotConfig.js` (+3 / -3)

Unrelated to the Views variant — a data-correctness fix committed in the same range (`dd84b57`). Percentage-column denominators were wrong:

Column	Before (<code>targetField</code>)	After
Target %	<code>target</code>	<code>target_value</code>
Prod %	<code>target</code>	<code>net_primary</code>
Inv %	<code>target</code>	<code>net_primary</code>

`Target %` was pointing at a field name that no longer matched the query output. `Prod %` and `Inv %` were being divided by `target` when the intended denominator is `net primary` — offer percentages are a share of actual primary sales, not of target. `beforeColumn` for Target % moved to `target_value` to match.

5. Plasmic Studio prop reference

Elbrit DataProvider (Views)

Prop	Type	Default	Notes
<code>views</code>	object	Cards + Table	<code>['Cards', 'Table']</code> or <code>[{id,label,icon}]</code> ; ids must match <code>DataRow.viewId</code>
<code>defaultView</code>	string	first view	
<code>activeView</code>	string (writable state)	—	unset = provider owns selection
<code>onViewChange</code>	eventHandler(<code>viewId</code>)	—	
<code>showViewSwitcher</code>	boolean	<code>true</code>	off → build your own, call <code>\$ctx.view.setActiveView(id)</code>
<code>viewSwitcherPosition</code>	choice	<code>header</code>	<code>header</code> <code>top</code> <code>bottom</code>
<code>viewSwitcherAlign</code>	choice	<code>right</code>	<code>top</code> / <code>bottom</code> only
<code>keepInactiveMounted</code>	boolean	<code>true</code>	keeps table scroll/expanded rows
<code>showSearch</code>	boolean	<code>false</code>	needs <code>clientSave: true</code> + <code>searchFields</code>
<code>searchPlaceholder</code>	string	<code>Search product or brand...</code>	
<code>showRecentSearches</code> / <code>recentSearchLimit</code> / <code>recentSearchStorageKey</code>	bool / num / string	<code>true</code> / <code>5</code> / —	set the key to isolate per page
<code>compactHeader</code>	boolean	<code>= showSearch</code>	replaces engine controls with pills
<code>hideNativeFilterSort</code>	boolean	<code>false</code>	
<code>showLetterRail</code>	boolean	<code>false</code>	needs <code>[data-letter]</code> in the slot
<code>letterRailField</code>	string	—	e.g. <code>brand__name</code> ; enables live dimming
<code>letterRailViews</code>	object	<code>['cards']</code>	<code>[]</code> = all views
<code>staleWhileRevalidate</code>	boolean	<code>false</code>	<code>\$ctx.data.main.isRevalidating</code> during stale window
<code>cacheKey</code>	string	<code>preset:{src}: {name}</code>	set to unshare snapshots
<code>presetDataSource</code> , <code>presetName</code> , <code>offlineData</code> , <code>overrides</code> , <code>onDataChange</code> , <code>onError</code>	—	—	identical to Elbrit DataProvider

Elbrit DataView

Prop	Type	Notes
<code>viewId</code>	string	must match an id in the parent's <code>views</code>
<code>keepMounted</code>	boolean	overrides parent's <code>keepInactiveMounted</code>
<code>className</code> / <code>children</code>	string / slot	setting <code>className</code> opts out of <code>display: contents</code>

Studio bindings

- `$ctx.data` — unchanged (data, columns, filter/sort state)
- `$ctx.view` — `{ views, activeView, setActiveView, isActive, keepInactiveMounted }`

6. Backward compatibility

File	Guarantee
<code>DataProviderNew.jsx</code>	<code>headerSlots</code> null and <code>hideNativeFilterSort</code> false $\Rightarrow$ header condition, <code>selectorsJSX</code> gating, and sidebar mode all evaluate to the pre-change values. The six new context entries are additions only.
<code>FilterSortSidebar.jsx</code>	<code>sortOnly</code> defaults <code>false</code> $\Rightarrow$ original tabs, header text, clear behaviour.
<code>plasmic-init.js</code>	Registrations added; <code>DataProvider</code> and <code>DataTableNew</code> metas untouched.
<code>DataProvider.jsx</code>	Not modified — the variant reuses it as-is.

No existing page passes any of the new props. Nothing currently rendered changes.

7. Known limitations and gotchas

1. **Compact mode hides the month picker.** `compactHeader` sets `showProviderHeader: false`, which zeroes `hasHeaderContent` and drops all of `selectorsJSX`. Month-range queries need the engine header — keep `compactHeader` off for those. (Documented in the Studio prop description.)
2. **Compact mode also hides applied-filter chips**, since they live inside `selectorsJSX`. Note the distinction: `hideNativeFilterSort` keeps the chips; `compactHeader` does not. If chips matter on a compact screen, `hideNativeFilterSort` alongside the standard header is the combination to use.
3. **Search silently no-ops without `clientSave: true` + a `searchFields` map** on the query doc. The input shows an explanatory `title` tooltip rather than being disabled — check the query doc first when search "does nothing".
4. **The letter rail needs `[data-letter]` targets in the slot.** With no matching sections, the letters dim and clicks do nothing. Custom Studio layouts must add the attribute themselves.
5. **`staleWhileRevalidate` on editable tables is misleading.** During the stale window the view is a static picture of the previous session; interactions apply to the live provider and take effect when fresh data lands.
6. **Snapshots are skipped above 20,000 rows** — large datasets fall back to the normal spinner with no warning surfaced in the UI.
7. **The DOM contract is untyped.** A typo in `data-letter` or a renamed `letterRailField` fails silently rather than raising. Both are the cost of decoupling the rail from Studio-authored layout.

8. Commit map

Commit	Change
62ee1ee	<code>DataProviderViews</code> , <code>DataView</code> , <code>ViewContext</code> + registration — tabs working
7553e84	<code>ProductSearchBar</code> , <code>SortSheet</code> (first sort attempt), header-slot plumbing
2d911d7	<code>AlphabetRail</code> added
a050de4	<code>SyncPill</code> , <code>StaleDataBridge</code> ; rail + search refinements
40f5216	<code>SortSheet</code> deleted , <code>FilterSortPill</code> added ; sort-only mode in <code>FilterSortSidebar</code>
2198421	Rail: scroll-tracked active letter
d678016 — 3e7d5ec	Pill sizing/spacing, single-row compact header, fixed-position popovers
dd84b57	<code>slotConfig.js</code> percentage-denominator fix (unrelated)

Companion work outside [share/](#) (context only)

The same range adds root-level catalog components — `components/ProductCard.jsx` , `CatalogLetterGroup.jsx` , `CatalogLetterSection.jsx` , `ProductStockSheet.jsx` — plus `KT.md` and `erp-queries-inventory.md` . These are the **consumers** of the variant: `CatalogLetterGroup` is what renders the `data-letter` sections the rail scrolls to. They are a separate surface and are not covered by this document.